Project Name: PathMate.ai

Team Members:

- **Dohyeon Lee** (dohyeon.lee@stonybrook.edu)
- **Heesang Kim** (heesang.kim@stonybrook.edu)

1. Data Identification

Input Data

- **User Input:** The system collects basic personal details directly from users things like their name, email, major, GPA, and technical skills. Nothing unusual here, just the kind of info you'd expect when building a profile.
- **File Input:** Users can also upload resumes or portfolios (PDF format). This helps give a fuller picture of their experience beyond just form inputs.
- **External Data:** On top of that, the system pulls job posting data from external sources like job boards and company websites. This includes role descriptions, requirements, and other relevant hiring details.

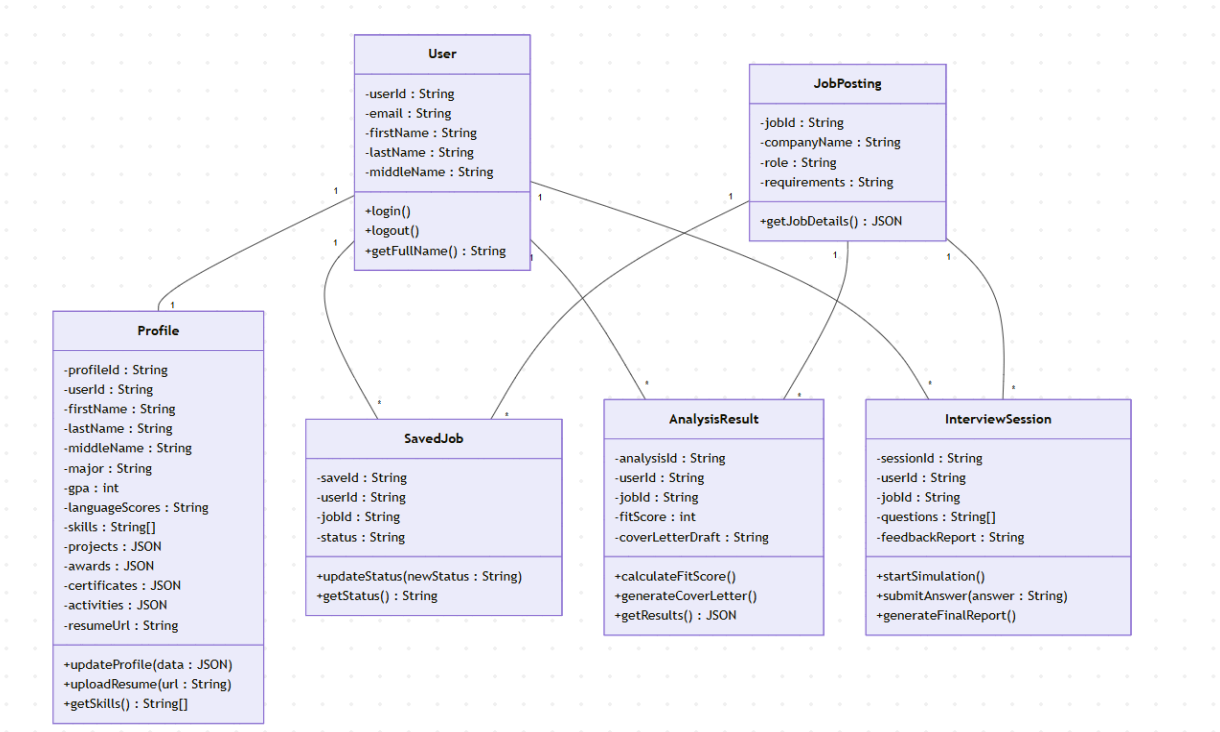
Output Data

- **Analysis Results:** The system calculates a “fit score” (basically a percentage) that estimates how well a user matches a job. It also explains why not just a number, but reasoning behind it..
- **Generated Documents:** It can generate tailored cover letter drafts for specific roles and suggest improvements to the user’s resume. Think of it more like guidance rather than just auto-writing everything..
- **Interview Insights:** There’s also a mock interview feature (Hiring Simulator) that gives feedback reports and practical suggestions based on how the user responds.

2. Data Modeling: UML Class Diagram & Data Schema (SQL)

This UML class diagram outlines the core data structure behind PathMate.ai. These classes basically act as the bridge between the database (Supabase) and the Next.js frontend.

(a) UML Class Diagram



Dictionary

- *firScore* from AnalsusResult class : A percentage score generated by AI that represents how well the user's profile aligns with a job posting.
- *status* from SavedJob class : Tracks where the user is in the application process—like 'saved', 'applying', 'interviewing', or 'completed'.
- *JSON* types from projects, awards, etc.. : These are stored as flexible JSON types so we can handle more complex or unstructured data (e.g., projects, roles, tech stack).

(b) Data Schema (Next.js)

```
type Json = string | number | boolean | null | { [key: string]: Json } | Json[];

export interface User {
  userId: string;
  email: string;
  firstName: string;
  lastName: string;
  middleName?: string;
}

export interface Profile {
  profileId: string;
  userId: string;
  firstName: string;
  lastName: string;
  middleName?: string;
  major: string;
  gpa: number;
  languageScores: string;
  skills: string[];
  projects: Json;
  awards: Json;
  certificates: Json;
  activities: Json;
  resumeUrl: string;
}

export interface JobPosting {
  jobId: string;
  companyName: string;
  role: string;
  requirements: string;
}

export interface SavedJob {
  saveId: string;
  userId: string;
  jobId: string;
  status: 'saved' | 'applying' | 'interviewing' | 'completed';
}
```

```
export interface AnalysisResult {
  analysisId: string;
  userId: string;
  jobId: string;
  fitScore: number;
  coverLetterDraft: string;
}

export interface InterviewSession {
  sessionId: string;
  userId: string;
  jobId: string;
  questions: string[];
  feedbackReport: string;
}
```

3. Persistence (Plan to use ORM mapping)

We're using the *Supabase-js client* along with TypeScript to connect everything. One nice thing about Supabase is that it can automatically generate types based on the database schema, which helps keep things consistent between the backend and frontend.

Instead of writing complex SQL queries all the time, we rely on Supabase's client library. It handles database operations, authentication, and even file storage through simple function calls which makes development a lot smoother.

4. Contributions

Dohyeon Lee :

- Worked mainly on the frontend using Next.js and Tailwind CSS. He built out the UI for the homepage, profile pages, and dashboards.
- He also implemented job filtering features and parts of the mock interview interface.
- For web view evaluation, dummy data with a structure similar to that of real service environments were built and integrated into the system.

Heesang Kim :

- Focused more on the backend/data side. He designed the UML diagram and database schema.
- He also handled GitHub repo management and deployment through Vercel, and set up the initial Supabase structure.
- In addition, he finalized the data design report and updated the README with setup instructions.